Sprog og oversættere

Lexical analysis: Regular expressions and NFAs

Part 5: From regular expression to NFA

Hans Hüttel

(using content by Cosmin Oancea)

Converting a regular expression to an NFA

There is a simple algorithm for converting a regular expression to an equivalent NFA. The algorithm has a case for each form of expression.

Each expression produces an NFA *fragment* that has an incoming half-transition which is not and an outgoing half-transition labelled by either ϵ or by an alphabet symbol.

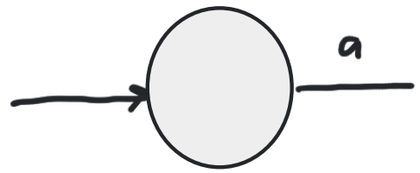
Converting a regular expression to an NFA

There is a simple algorithm for converting a regular expression to an equivalent NFA. The algorithm has a case for each form of expression.

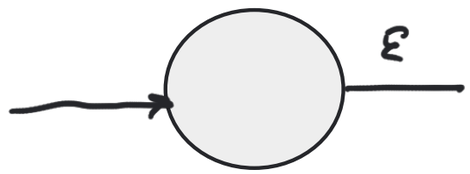
Converting a regular expression to an NFA

There is a simple algorithm for converting a regular expression to an equivalent NFA. The algorithm has a case for each form of expression. First we consider atomic expresssions.

$R = a$



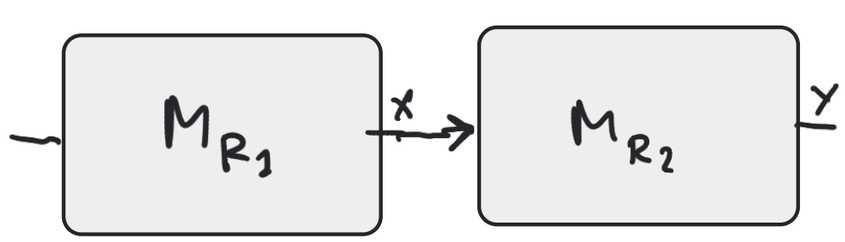
$R = \varepsilon$



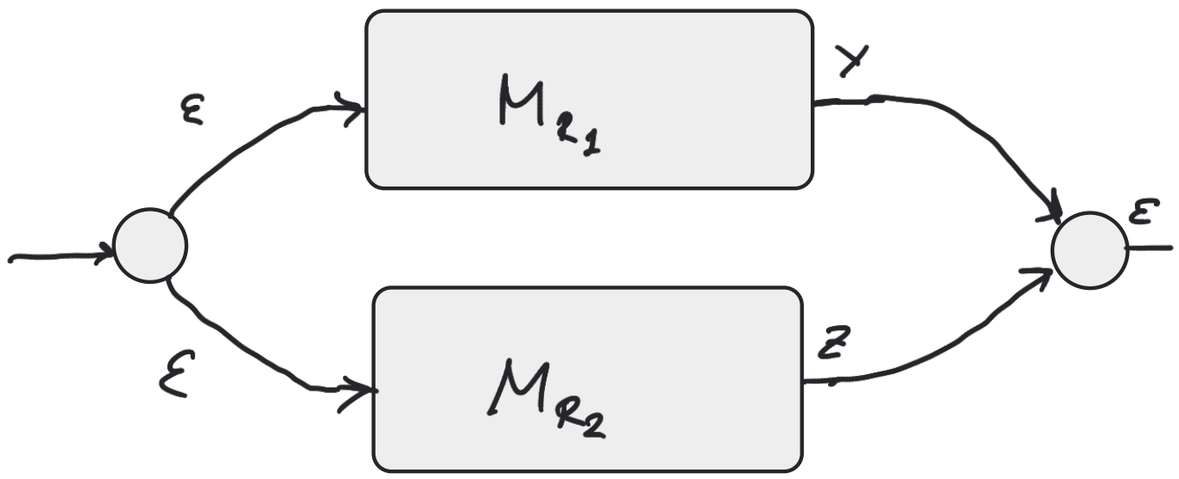
Converting a regular expression to an NFA

Now we consider composite expressions. Here we join the fragments by their half-transitions. Assume that we already have NFA fragments for the languages described by regular expressions R_1 and R_2.

$R = R_1 R_2$



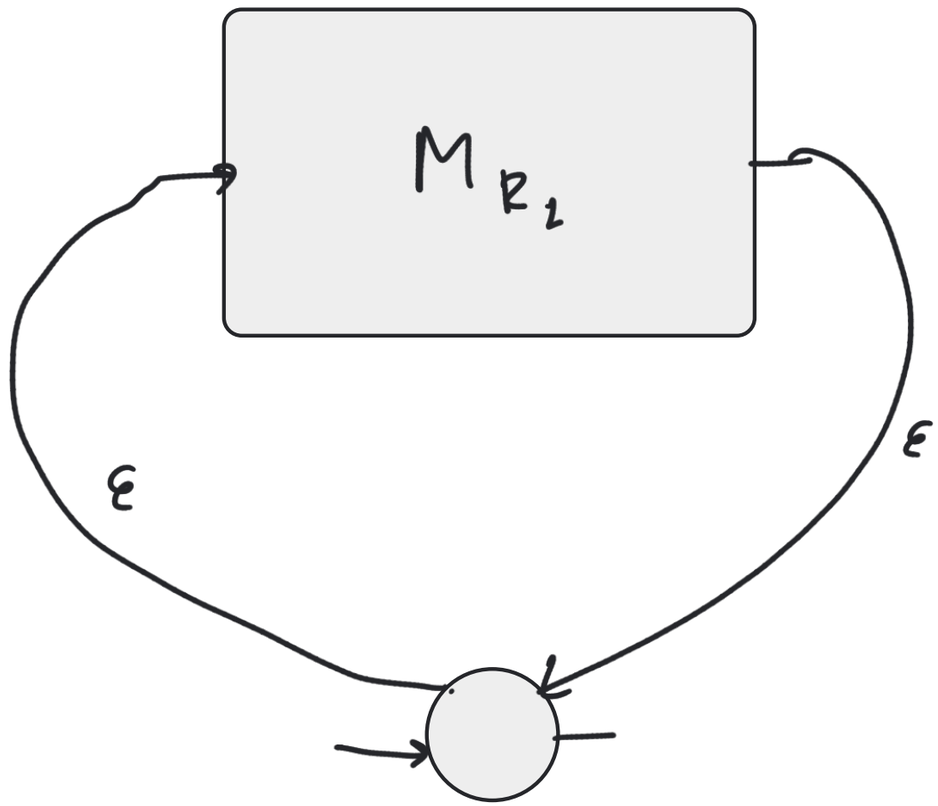
$R = R_1 \mid R_2$



Converting a regular expression to an NFA

This is the star of the show! Assume that we already have an NFA that recognizes the language described by R.

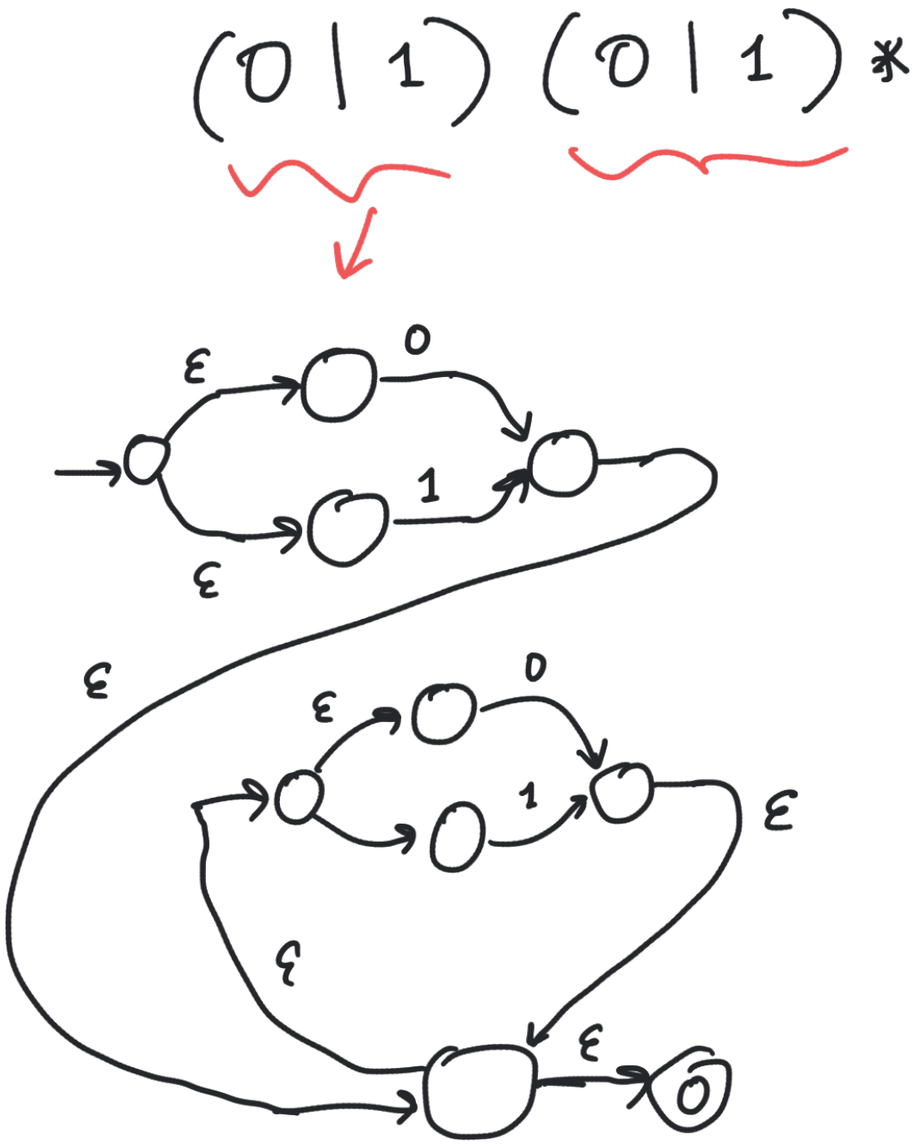
$R = R_1^*$



Finishing the construction

Finally, when an NFA fragment has been constructed for the whole regular expression, we connect the outgoing half-transition to an accepting state. The state with the incoming half-transition becomes our initial state.

An NFA for binary numbers



Getting rid of nondeterminism

- The ϵ transitions can cause the NFA to change state without reading a symbol
- The NFA also makes guesses when it reads a symbol

One cannot implement that. How can we get rid of it?

Differences from Sipser's algorithm

The conversion algorithm one sees in Sipser's textbook in Section 1.2 is a little different.

The construction in this course

- builds NFA fragments, not NFAs, and the cases for composite regular expressions tell us how to connect such fragments, and
- creates an NFA with one accepting state. This is useful when we want to build a lexer later on.

Sipser's construction builds a full NFA for each sub-expression and connects them with ϵ-transitions. Moreover, we can now have more than one final state. And ...

Differences from Sipser's algorithm

The conversion algorithm one sees in Sipser's textbook in Section 1.2 is a little different.

The construction in this course

- builds NFA fragments, not NFAs, and the cases for composite regular expressions tell us how to connect such fragments, and
- creates an NFA with one accepting state. This is useful when we want to build a lexer later on.

Sipser's construction builds a full NFA for each sub-expression and connects them with ϵ-transitions. Moreover, we can now have more than one final state. And ... since one can use $\emptyset$ as a regular expression, it becomes possible to build NFAs without accept states.

Getting rid of nondeterminism

- The ϵ transitions can cause the NFA to change state without reading a symbol
- The NFA also makes guesses when it reads a symbol

One cannot implement that. How can we get rid of it? There is also an algorithm for that!

Getting rid of nondeterminism

- The ϵ transitions can cause the NFA to change state without reading a symbol
- The NFA also makes guesses when it reads a symbol

One cannot implement that. How can we get rid of it? There is also an algorithm for that!

But that will have to wait until next time.